CS673 Software Engineering
SafeDrop
Software Design Document

Your project Logo here if any

Team Member	Role(s)	Signature	Date
Alexa Stein	Team Lead	*Alexa Stein*	10/12/2026
Amber Rastella	Security Lead	*Amber Rastella*	10/12/2026
Alex Picard	Configuration Lead	*Alex Picard*	10/12/2026
Mateus Silva	Requirements Lead	*Mateus Silva*	10/12/2026
Orelmis Toribio	QA Lead	*Orelmis Toribio*	10/12/2026

Revision history

Version	Author	Date	Change
0.1	Amber Rastella	9/16/2026	Initial draft of Sections 1, 2, 6, 8, 10, 12, and 13 from team materials (AI-assisted); pending team review
0.2	Amber Rastella	9/21/2026	Iteration 1 verification pass against the merged code (PRs #41-#45). Recorded the settled design decisions in Section 10.1: OD-1 Option A, OD-2, OD-3 and OD-6 Decided; OD-5 and OD-7 decided subject to team confirmation. Corrected Sections 2.3.4 (staging deploys via a mirror repository; production is still a placeholder), 6.2 (logout revokes

			rather than deletes; the role re-read is narrower than stated) and 6.6 (the insert-only database role is defined but not yet applied, so SR-8 rests on the application layer). Completed the audit action and target-type lists. Wrote Section 3 (domain model and package structure, Figures 3-4), Section 4 (route map, screen inventory and screenshots, Figures 5-11), Section 5 (Database Design), Section 7 (checkout state machine) and Section 9 (REST APIs) from the implemented code. Sections 3 and 4 are no longer outstanding.
0.3	Orelmis Toribio	10/11/2026	Iteration 2 design changes from SCRUM-148 (configurable approval) and SCRUM-205 (two-sided custody); closes SCRUM-152. Extended Section 8.5 to show how the configurableApproval policy and canConfirmReturn were added without changing the approve service; extended the Section 7 state machine with

			EXPIRED and RETURN_PENDING and documented auto-approval at submission (7.1) and the two-step return and expiry (7.2); added approvalSettings, pickupSettings, approvalMode and the return-report fields to Section 5; updated the Section 6.4 permission matrix, the Section 6.6 audit action list, the Section 9 route table, the Section 4 screen notes and OD-4.
0.4	Mateus Silva	10/12/2026	Updated application screenshot figures under Class Diagrams section

● Introduction
● Software Architecture
● Class Diagram
● UI Design (if applicable)
● Database Design (if applicable)
● Security Design
● Business Logic and/or Key Algorithms
● Design Patterns
● Rest APIs
● Any Additional Topics you would like to include.
● AI usage Log
● References
● Glossary

(In your submitted document, please delete all instructions)

1. Introduction

1.1 Purpose and Scope

This Software Design Document (SDD) describes the design of SafeDrop, a multi-tenant web application that lets organizations lend physical assets to their members through request, approval, checkout, and return workflows while recording every action in an immutable audit history. It complements the Software Project Proposal and Plan (SPPP) [1], which defines what SafeDrop will do, by describing how the system is structured and why each major design decision was made.

This is a living document that is updated at the end of each iteration. The current version reflects the design for Iteration 1, which delivers the essential features: organization portal creation, login and authentication, asset checkout and return with a fixed approval step, the audit log, and the admin dashboard. Design elements intended for Iterations 2 and 3 are explicitly labeled as planned, so that readers can distinguish the design being built now from extension points reserved for later. Two scope notes for this version. First, a Microsoft Foundry client and its configuration were merged during Iteration 1 as groundwork for Iteration 3; it is disabled by default, no route or service calls it, and it is not an Iteration 1 deliverable. Second, SR-8 (audit immutability) is only partly satisfied: the application layer is implemented and tested, the database layer is defined but not yet applied. Section 6.6 gives the detail.Version 0.3 adds the first Iteration 2 design changes: approval rules that an Org Admin configures per organization and per asset (Section 8.5), and two-sided custody, in which pickup can be recorded by the borrower, returns are confirmed by someone other than the borrower, and uncollected approvals expire (Sections 7 and 8.5). Version 0.3 adds the first Iteration 2 design changes:

approval rules that an Org Admin configures per organization and per asset (Section 8.5), and two-sided custody, in which pickup can be recorded by the borrower, returns are confirmed by someone other than the borrower, and uncollected approvals expire (Sections 7 and 8.5).

1.2 Intended Audience

The primary audience is the SafeDrop development team, who implement the system against this design and cite it during code review. The secondary audience is the course instructor and facilitator, who evaluate the design and its traceability to requirements.

1.3 Conventions

Security and nonfunctional requirements are cited by the stable identifiers defined in the Nonfunctional Requirements document [2]: **SR-#** for security requirements and **NFR-#** for all other nonfunctional requirements. Functional requirements are referenced by feature name. Design decisions that the team has not yet finalized are labeled **OD-#** (open decision) and tracked in Section 10.1, so that unresolved choices remain visible rather than being settled silently.

1.4 Design Goals

The following goals guide the design. Where goals conflict, G1 through G3 take precedence over developer convenience and raw performance, because a failure in tenant isolation, audit integrity, or inventory consistency would undermine the product's core value proposition.

Goal	What it means for the design	Driven by
G1. Tenant isolation by construction	The organization ID is taken only from the verified session and is required by every data-access function, so cross-tenant access cannot occur without deliberately bypassing the data-access layer.	SR-2, NFR-4
G2. Trustworthy audit history	Audit entries are written atomically with the state change they describe, and the application has no ability to modify or delete them.	SR-8, SR-9, SR-10
G3. Consistent inventory state	State transitions are atomic and conditional, so an asset is never left partially checked out after a failure.	NFR-2
G4. Server-side enforcement	Authorization and input validation happen in the API. Restrictions in the user interface are a usability convenience, never a security control.	SR-1, SR-6
G5. Testability and modularity	A layered backend with narrow interfaces lets each layer be tested with its dependencies mocked, supporting the QA targets of at least 80% line coverage and low fan-out per file.	NFR-10, NFR-11; SPPP QA plan
G6. Extensibility	Iteration 1 approval logic and data access are structured so that configurable approval workflows, user tiers, and AI features can be added in later iterations without restructuring existing components.	SPPP desirable and optional features

2. Software Architecture

2.1 Architectural Overview

SafeDrop uses a three-tier client-server architecture with a separately hosted frontend and backend. A React single-page application (SPA) is served by Vercel and runs in the user's browser. The SPA communicates exclusively with an Express REST API hosted on Render, over HTTPS using JSON. The API is the only component with access to the MongoDB Atlas database. Figure 1 shows the complete system, including the delivery pipeline and the planned Iteration 3 AI integration.

Two considerations drove this structure. First, separating the frontend and backend allows each to be developed, tested, and deployed independently, which suits a five-person team working in parallel on short-lived feature branches. Second, it concentrates every security decision in a single tier: the browser is treated as untrusted, so anything the SPA sends is validated and authorized by the API regardless of what the user interface allows (G4). The main cost of this structure is that the frontend and backend run on different domains, which complicates cookie-based authentication; that trade-off is analyzed, and resolved in favour of a same-origin proxy, in Section 6.3 (OD-1).

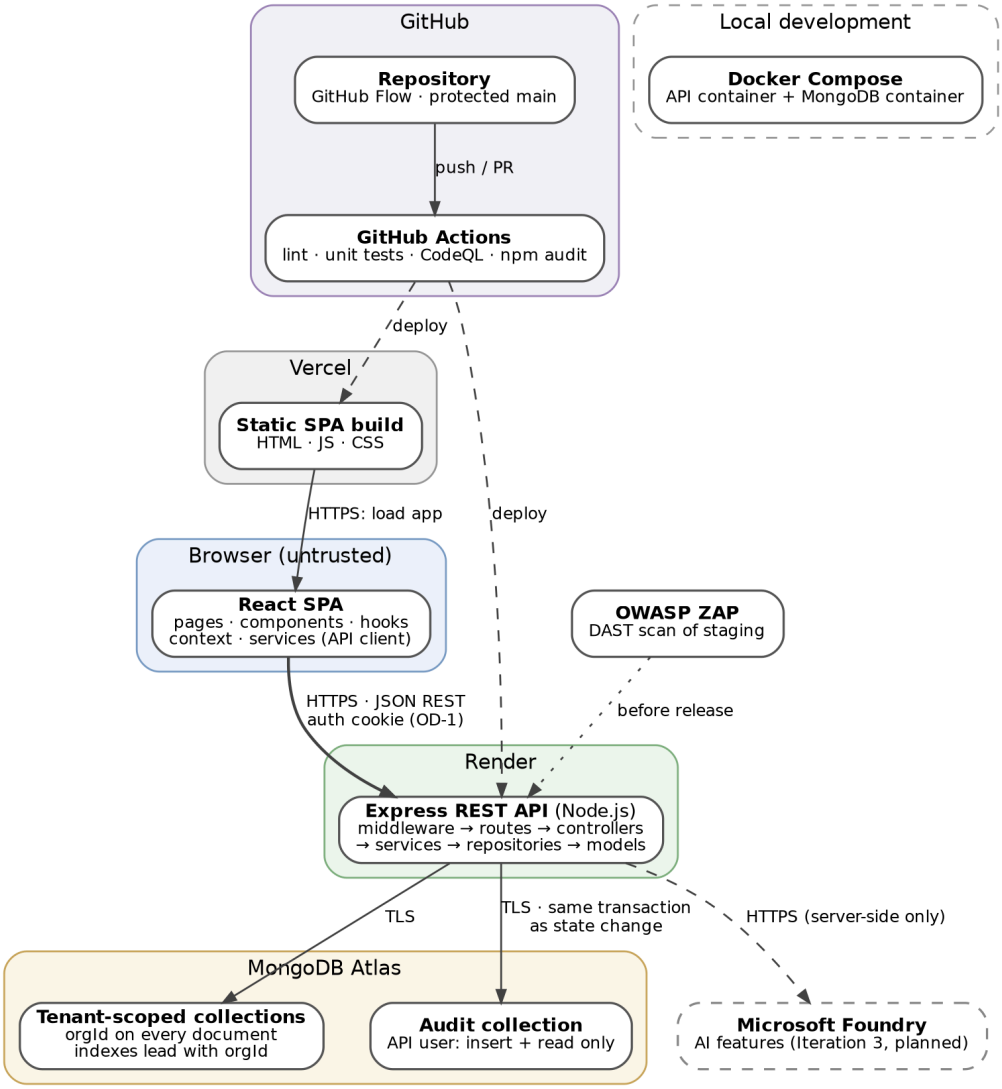


Figure 1. SafeDrop system architecture. Solid arrows are runtime traffic; dashed arrows are deployment or planned integrations; the dotted arrow is security testing.

2.2 Frameworks and Technologies

Concern	Technology	Role in SafeDrop
Frontend	React (JavaScript)	Single-page user interface for members, approvers, and admins
Backend	Node.js, Express	REST API and middleware pipeline
Data access	Mongoose ODM (OD-6)	Schema definitions, indexes, and queries used by the repository layer
Database	MongoDB Atlas	Managed document database; all tenant data and the audit log

Concern	Technology	Role in SafeDrop
Testing	Vitest, React Testing Library, Supertest; Playwright if time allows	Frontend and backend unit tests; API route tests without a running server; end-to-end tests
Containers	Docker, Docker Compose	Reproducible local environment (API, SPA and a MongoDB replica set); CI builds both images for parity; Render deploys the API from its Dockerfile (OD-5)
CI/CD and hosting	GitHub Actions, Vercel, Render	Automated checks on every push and pull request; staging and production deploys
Security tooling	GitHub CodeQL, npm audit, OWASP ZAP	Static analysis, dependency scanning, dynamic scanning of staging
AI (Iteration 3, planned)	Microsoft Foundry	Search recommendations, reliability scoring, depreciation projection. Iteration 1 merged the transport and configuration only (FOUNDRY_ENABLED defaults to false, and the app refuses to boot if it is switched on without an endpoint, key and deployment). There is no search service and no endpoint yet.

2.3 Component Decomposition

2.3.1 Frontend (React SPA)

The frontend is organized by responsibility, following the folder structure in the SPPP coding standard [1]. The key rule is that only `src/services` communicates with the API. Components and hooks never call `fetch` directly, which mirrors the backend repository rule and allows API calls to be mocked in component tests.

Folder	Responsibility	May depend on
`src/pages`	Route-level screens (for example, inventory browsing, my requests, admin dashboard, audit log)	components, hooks, context
`src/components`	Reusable presentational UI elements	components, utils
`src/hooks`	Reusable stateful logic such as loading inventory or submitting a request	services, context
`src/context`	Application-wide state: the signed-in user, role, and organization	services
`src/services`	The only module that calls the API; one function per endpoint	utils
`src/utils`	Pure helpers such as formatting and constants	none

The frontend hides controls that a user's role cannot use, but this is purely a usability choice. The API independently enforces every permission (SR-1).

2.3.2 Backend (Express API)

The backend is a layered architecture (Section 8.1). Each layer has one responsibility and may depend only on the layers beneath it; lower layers never import from higher

ones. This rule keeps fan-out low, lets services be unit-tested with mocked repositories, and lets routes be tested end-to-end through Supertest.

Layer (folder)	Responsibility	Depends on	Must not
`src/app.js`, `src/server.js`	Assemble middleware and routes; start the HTTP server	all layers	-
`src/middleware`	Cross-cutting concerns: security headers, CORS, rate limiting, authentication, tenant scoping, authorization, validation, error handling	utils; services for session lookups	Contain business rules
`src/routes`	Map each HTTP method and path to its middleware and controller	middleware, controllers	Access the database
`src/controllers`	Translate between HTTP and services: read validated input, call a service, set status and response body	services	Query the database or contain business rules
`src/services`	Business rules and workflows: approval step, checkout and return transitions, audit writing; own transactions	repositories, utils	Read Express request or response objects
`src/repositories`	All database operations; every function requires an organization ID	models	Contain business rules
`src/models`	Schema definitions and indexes	-	-
`src/utils`	Pure helpers: error types, logging, token utilities	-	-

One consequence is worth noting for the team. The SPPP coding standard uses `getUser(userId)` as its example repository function. Under this design, the signature becomes `getUser(orgId, userId)`, because tenant scoping is enforced at the repository boundary (Section 6.4).

2.3.3 Database (MongoDB Atlas)

Every tenant-owned document carries an `orgId` field, and every index used for tenant queries begins with `orgId`, which keeps per-organization queries efficient as the number of organizations grows (NFR-4). Because `orgId` partitions the data naturally, it is also the intended shard key if horizontal sharding is ever required. Sharding is not part of the current deployment, however, because the team runs on the Atlas free tier (OD-7), which does not offer it. The free tier also has no built-in backups, so a scheduled database export job provides the daily backup required by NFR-3. The audit log is stored in its own collection with restricted database privileges (Section 6.6). The detailed schema will be documented in Section 5.

2.3.4 Environments and Delivery Pipeline

Code moves through three environments, following the CI/CD plan in the SPPP [1]. Each environment uses its own database and its own secrets, so tests run in staging can never read or alter production data.

Environment	Where it runs	How it is updated	Purpose
Local	Developer machine via Docker Compose (API container and MongoDB container)	Manually by the developer	Development and running tests against a disposable local database
Staging	Vercel and Render staging services	Automatically after CI passes on main: the CI workflow mirrors main to a second repository that Render and Vercel watch (they do not watch this repository)	Integration testing and OWASP ZAP scanning
Production	Vercel and Render production services	Intended: tagged release at the end of each iteration. Not yet in place - the production deploy job in CI is still a skipped placeholder (Iteration 1)	Iteration demos and customer use

Every push and pull request runs linting, formatting checks, unit tests, CodeQL, npm audit and a Docker build of both images in GitHub Actions. A pull request also requires approval from a second team member before it can merge into the protected main branch.

2.4 Component Interfaces and Dependencies

Interface	Provider to consumer	Protocol and format	Notes
Application delivery	Vercel to browser	HTTPS; static HTML, JS, CSS	Build output of the React app
SafeDrop REST API	Express API to React SPA	HTTPS; JSON request and response bodies; authentication via httpOnly cookies	Endpoints will be specified in Section 9. Cookies are first-party through a Vercel proxy (OD-1, Option A)
Data access	MongoDB Atlas to backend repositories	MongoDB driver over TLS	Connection string supplied by environment variable (SR-5, SR-11)
Deployment	GitHub Actions to Vercel and Render	Platform integrations or deploy hooks	Credentials stored as GitHub Actions secrets
AI service (Iteration 3, planned)	Microsoft Foundry to backend services	HTTPS API	Called only from the backend; keys never reach the browser

2.5 Request Processing Pipeline

Every API request passes through the same ordered middleware chain before reaching a controller, as shown in Figure 2. Because tenant scoping and authorization sit in the request path itself, no individual route can accidentally skip them.

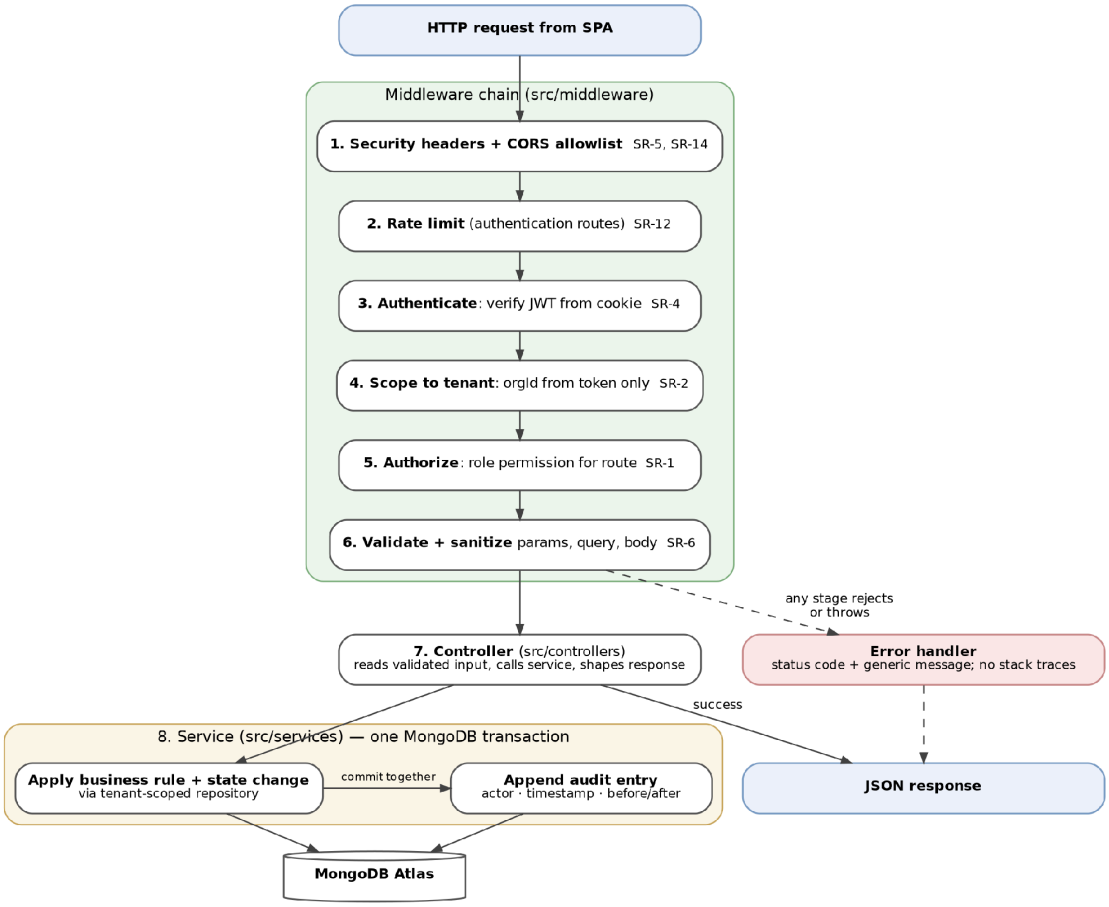


Figure 2. Request processing pipeline for an authenticated, state-changing API request.

1. **Security headers and CORS.** Standard security headers are applied to every response, and requests from origins outside the allowlist are rejected (SR-5, SR-14).
2. **Rate limiting.** Authentication routes are rate-limited to slow brute-force and credential-stuffing attempts (SR-12).
3. **Authenticate.** The access token is read from its cookie, and its signature and expiry are verified. The user ID, organization ID, and role are attached to the request context (SR-4). Login and token-refresh routes skip stages 3 through 5.
4. **Scope to tenant.** The organization ID from the verified token becomes the only tenant identifier for the request. Any organization ID supplied in the URL or body is ignored (SR-2).
5. **Authorize.** The user's role is checked against the permission declared for the route. Routes without a declared permission are denied by default (SR-1).
6. **Validate and sanitize.** Path parameters, query parameters, and the body are checked against the route's schema, and unknown fields are rejected (SR-6).
7. **Controller.** The controller reads the validated input and calls a service.

8. **Service.** The service applies business rules, performs the state change, and appends the audit entry in a single database transaction (SR-9, NFR-2).

Authentication runs before validation so that unauthenticated requests are rejected before the server spends effort parsing attacker-controlled input. Any stage that rejects a request or throws an error passes control to a central error handler, which returns an appropriate status code and a generic message without stack traces or database details.

Change from Iteration 0 (OD-2). The Iteration 0 presentation showed "write audit entry" as the final step of the middleware chain [3]. This design moves audit writing into the service layer. As a trailing middleware, the audit write would run after the state change had already been committed, so a crash between the two steps would leave a change with no audit record, violating SR-9 and NFR-2. The team confirmed this change (OD-2), and it is implemented: every service writes the audit entry inside the same transaction as the state change.

2.6 Extension Points for Later Iterations

- **Configurable approval workflows (Iteration 2, implemented for organization and asset level).** The approval rule sits behind a common policy interface (Strategy pattern, Section 8.5). SCRUM-148 used that interface as intended: an organization-wide default and a per-asset override were added as a new policy, configurableApproval, without changing the approve service. Per-equipment-group and per-user-tier rules remain future work and would be further policies behind the same interface.
- **User tiers and elevated-permission equipment (Iteration 2).** Authorization checks read from a central permission map, so new roles and tiers extend that map rather than adding conditionals to individual routes.
- **AI features (Iteration 3).** AI capabilities are added as a new service module that only the backend calls. Requests to Microsoft Foundry include only the minimum data required and never include names or email addresses (NFR-14).
- **TypeScript conversion (Iteration 3).** Because layers communicate through narrow module interfaces, files can be converted to TypeScript incrementally, one layer at a time.

3. Class Diagram

SafeDrop has two components with classes worth diagramming: the Express API and the React SPA. This section covers the domain model the API is built around, then the package structure of both components. Both diagrams are drawn from the Iteration 1 code rather than from the pre-implementation sketches, so every class, field and dependency shown can be checked against the repository.

3.1 Domain model (backend). The seven Mongoose models in code/backend/src/models are the application's entity classes. Three points in Figure 3 carry design weight. First, every tenant-owned class carries orgId, which is what makes tenant isolation checkable at the repository boundary (Section 6.4) rather than being a rule each query has to

remember. Second, Asset and AssetUnit are separate: an Asset is the catalogue entry ("a Dell XPS 15"), an AssetUnit is the individual physical copy that is actually lent, and a CheckoutRequest always points at a unit. Third, AuditEvent is the odd one out, marked here as append-only: every field is immutable, its repository exposes only append and query, and every mutating Mongoose hook throws (Section 6.6).

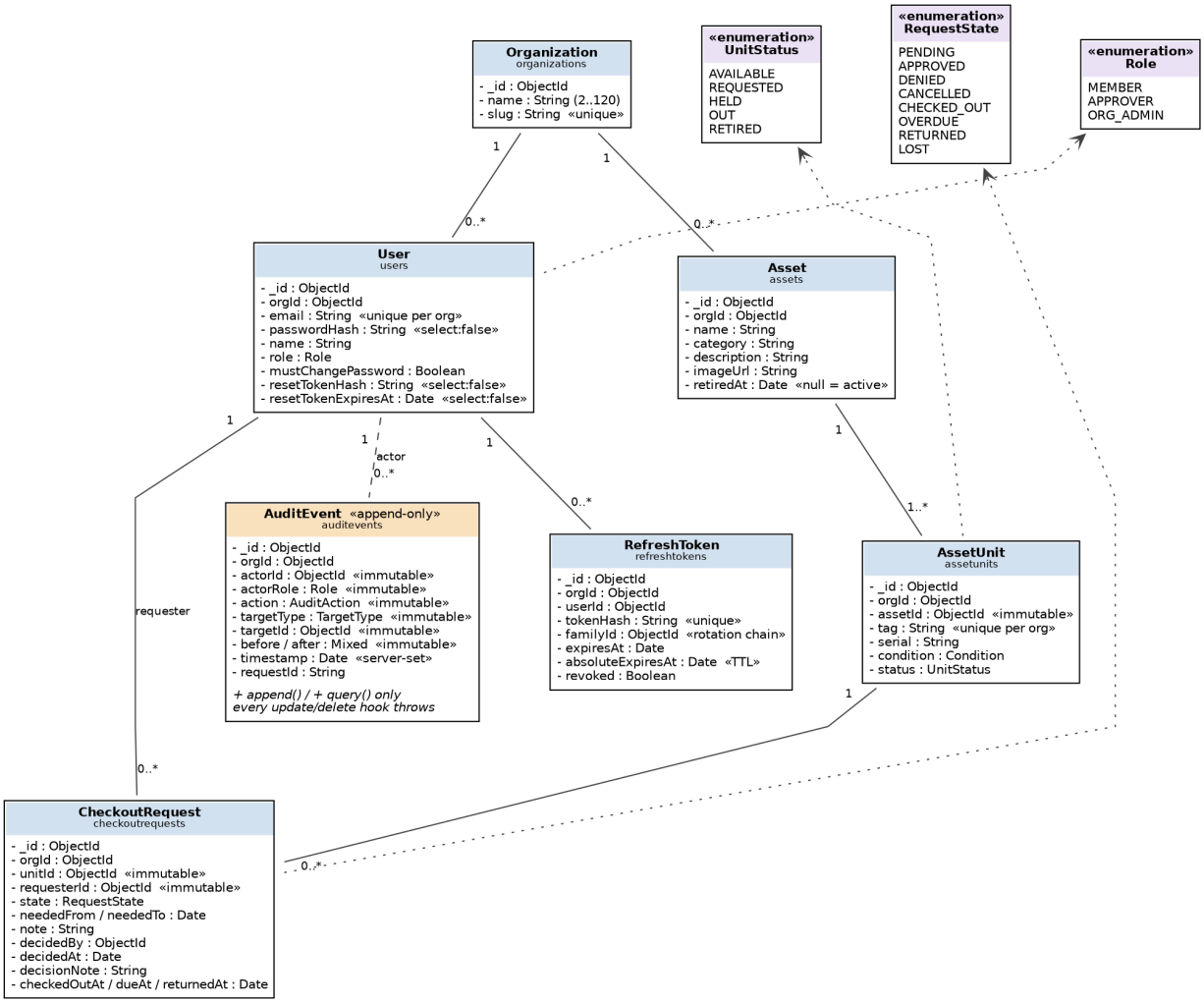


Figure 3. Domain model as implemented in Iteration 1. Fields shown are those defined in the Mongoose schemas; «unique» and «TTL» mark constraints created by the migrations. Dotted arrows are enumeration references. Fields added in Iteration 2 by SCRUM-148 and SCRUM-205 are listed in Section 5; the figure has not yet been regenerated.

3.2 Package structure (both components). Figure 4 shows the modules of each component and what each may depend on. The rule in both is the same and is the reason the structure is worth drawing: dependencies point one way only. In the backend a layer may use the layers beneath it and never above, so routes cannot reach the database and repositories cannot contain business rules. In the frontend only src/services calls the API, which ESLint enforces for components, pages, hooks and context. Two consequences the team relies on: any layer can be tested with the one beneath it mocked, and tenant scoping has exactly one place it can be got wrong.

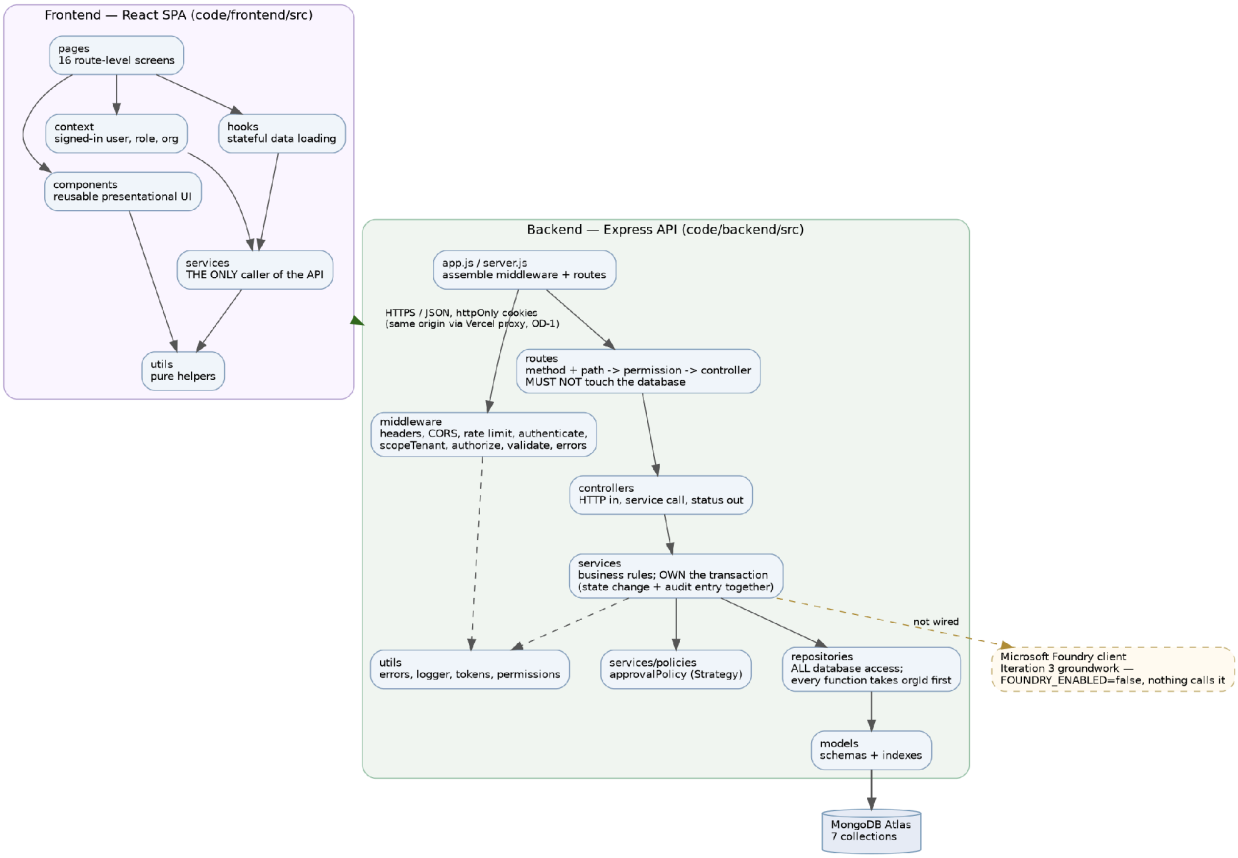


Figure 4. Component and package structure. Solid arrows are permitted dependencies; dashed arrows are shared utilities. The Foundry client is drawn detached because nothing calls it in Iteration 1.

4. UI Design (if applicable)

Iteration 1 ships sixteen screens. This section gives the route structure and what guards each part of it, an inventory of the screens, and screenshots of the main flows. One principle runs through all of it: the interface hides controls a role cannot use, but that is a usability convenience and never a security control - the API enforces every permission independently, and a member who types an admin URL gets the same refusal from the server as one who calls the endpoint directly (Sections 2.3.1 and 6.4).

4.1 Route structure and guards. Five screens are reachable without a session, because someone who cannot sign in has no session to guard. Everything else sits inside RequireAuth, then inside Layout (the shell with the navigation bar), and the administrative areas sit inside RequireRole on top of that. The forced password change sits inside RequireAuth but outside Layout, so an account still using a password somebody else chose cannot navigate anywhere until it picks its own.

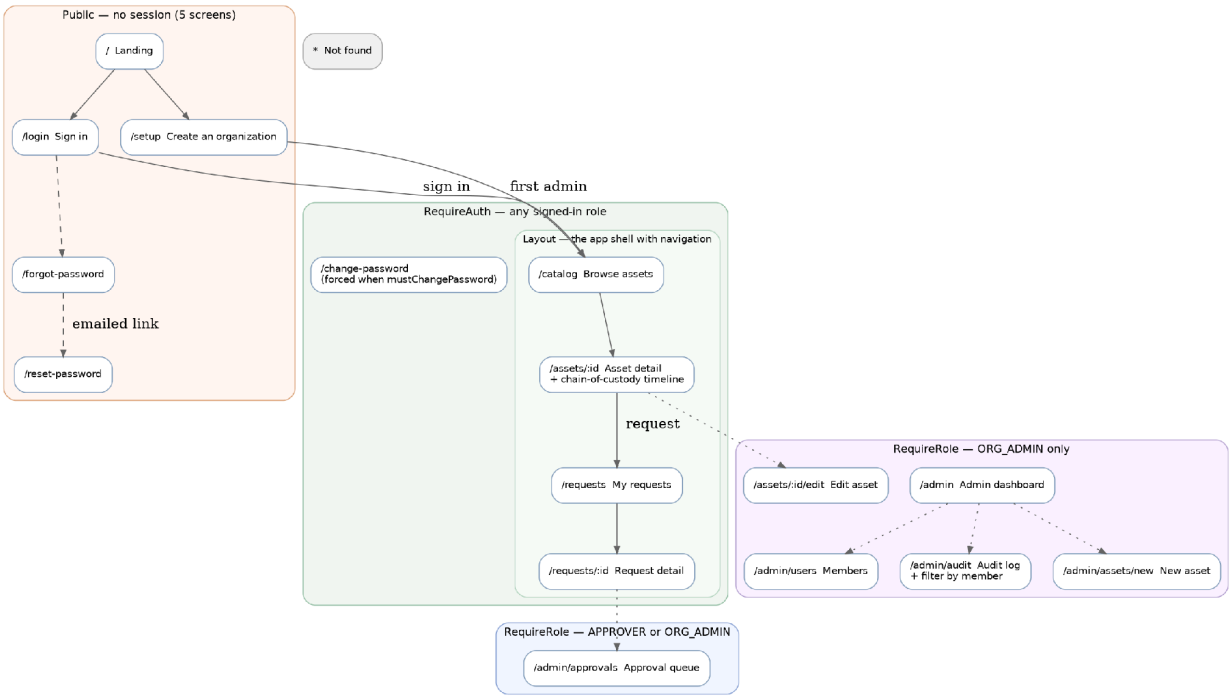


Figure 5. Route map and guard tiers. Solid arrows are the main navigation paths; dotted arrows are links from the navigation bar and inline actions.

4.2 Screen inventory.

Route	Screen	Who can reach it
/	Landing	Anyone
/login	Sign in (org slug, email, password)	Anyone
/setup	Create an organization and its first admin	Anyone
/forgot-password	Request a reset link	Anyone
/reset-password	Choose a new password from a mailed token	Anyone with the token
/change-password	Change own password; forced when mustChangePassword	Any signed-in user
/catalog	Browse the asset catalogue	Any signed-in user
/assets/:id	Asset detail, units, and the chain-of-custody timeline	Any signed-in user
/requests	My requests	Any signed-in user
/requests/:id	Request detail and its actions	Any signed-in user
/admin/approvals	Approval queue	Approver, Org Admin
/admin	Admin dashboard: counts and checkout activity	Org Admin
/admin/users	Members: invite, change role, reset password	Org Admin

/admin/audit	Audit log with member, action, target and date filters	Org Admin
/admin/settings	Organization settings: default approval mode (REQUIRED or AUTO) and pickup grace period in hours	Org Admin
/admin/assets/new	Create an asset	Org Admin
/assets/:id/edit	Edit an asset	Org Admin
*	Not found	Anyone

4.3 Screens. The captures below are the Iteration 1 interface as built. They were taken from the React application running in a browser, against the fixture dataset the frontend test suite uses (code/frontend/tests/mocks/handlers.js), whose response shapes mirror the real API contracts. They are therefore accurate as to layout, navigation and behaviour; the organization, people and assets in them are fixtures, not live data.

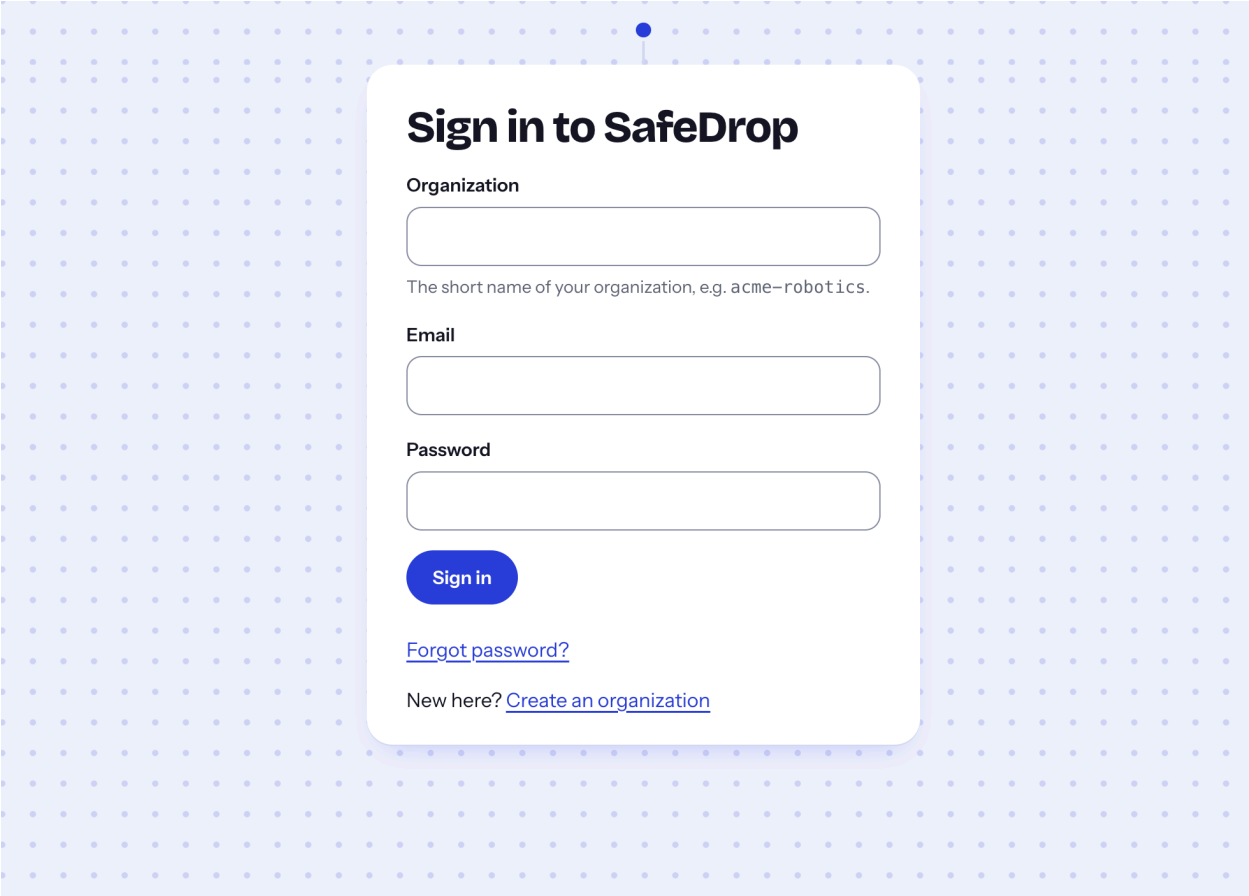


Figure 6. Sign in. The organization slug is part of the credential because email is unique per organization, not across SafeDrop (OD-3).

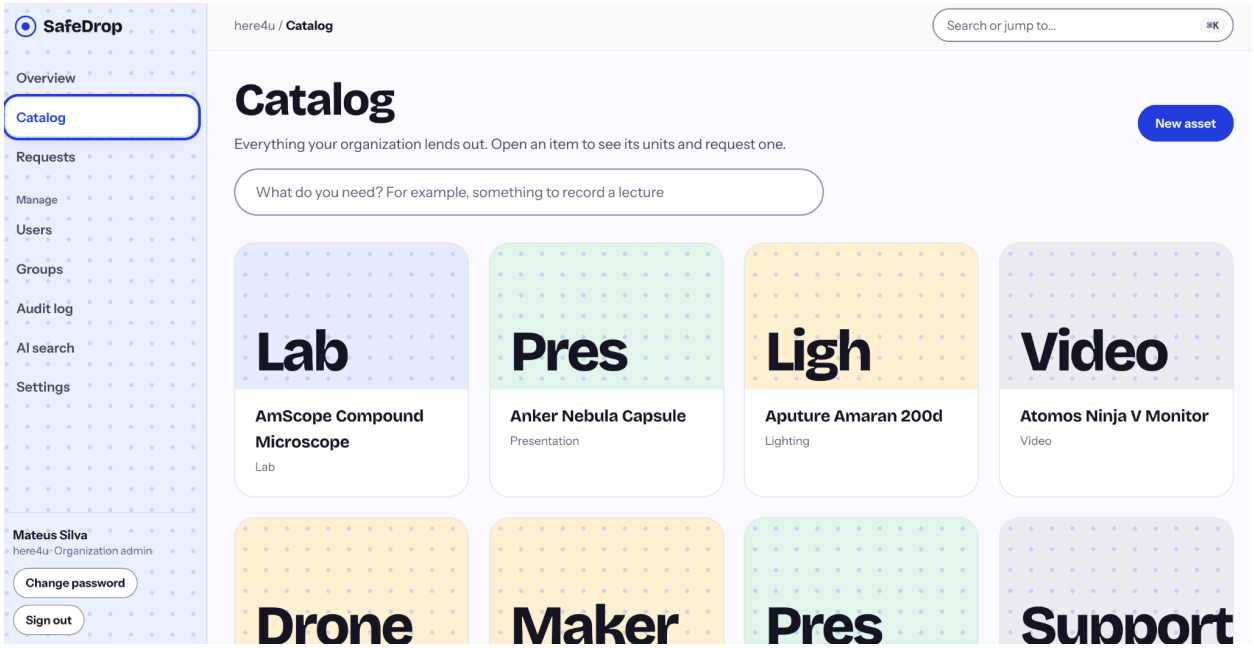


Figure 7. Catalogue. The first screen after signing in; every role sees it.



Figure 8. Chain of custody (SCRUM-29). One asset's history, unioned across the asset, its units and the requests against those units, with the actor and the role they held at the time. Behind audit:read rather than assets:read.

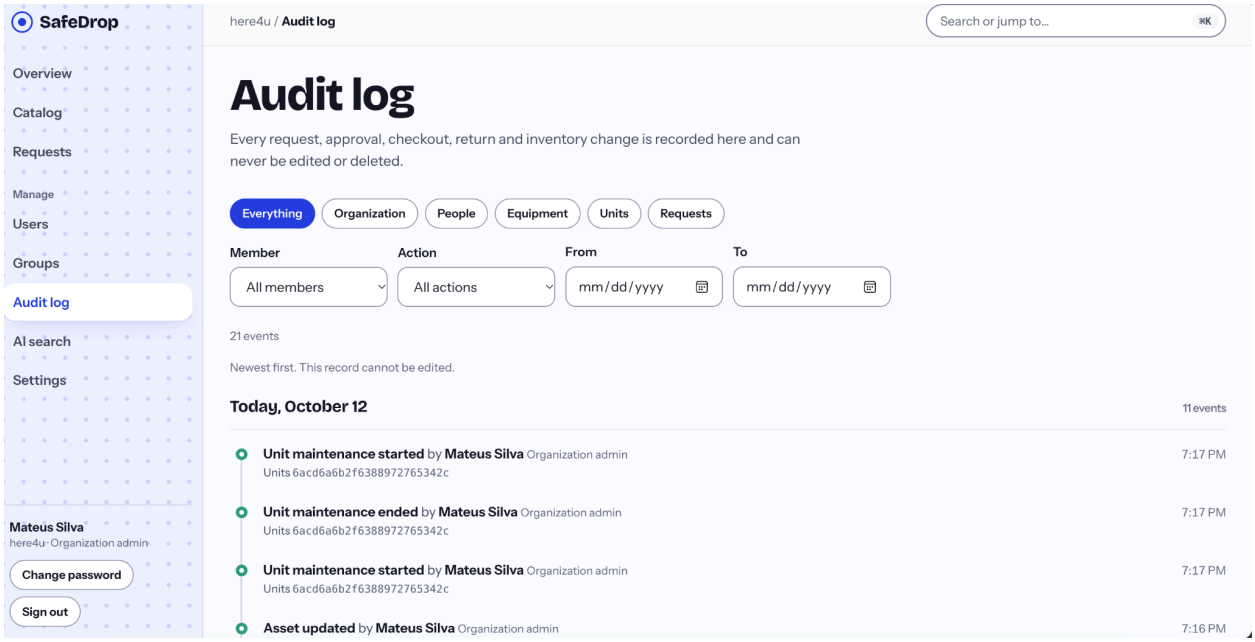


Figure 9. Audit log, Org Admin only. The Member control is the actor filter added in Iteration 1 (SCRUM-46): the backend had always supported filtering by actor, but no control existed, so acceptance criterion AT-1 was unreachable through the interface.

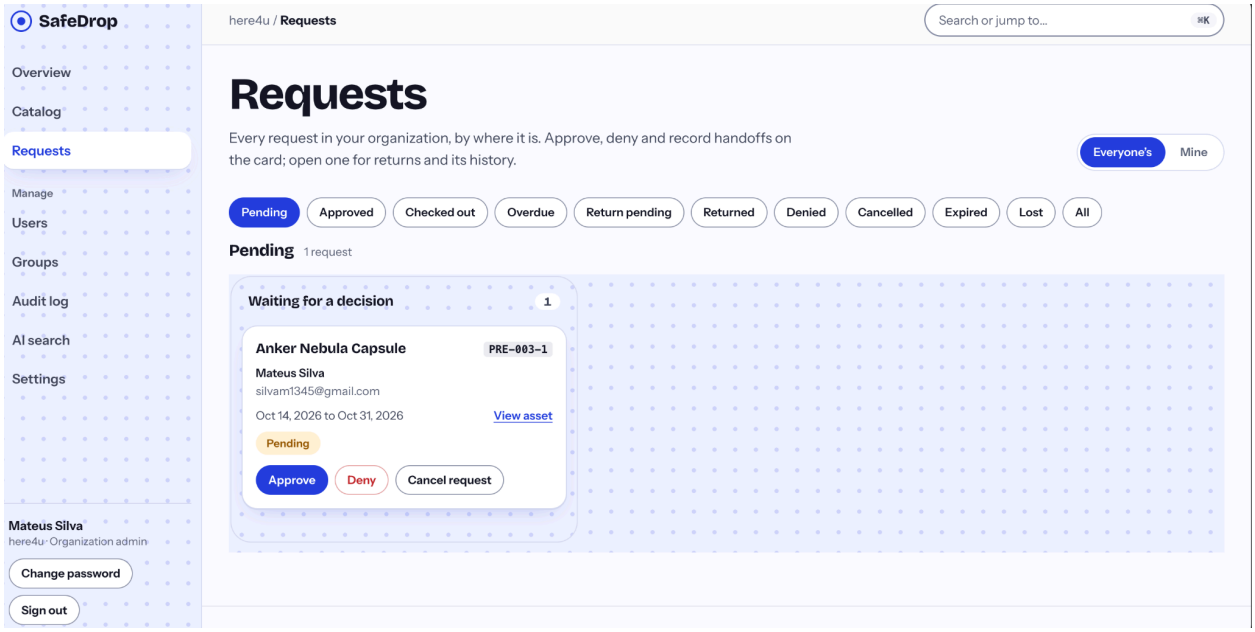


Figure 10. Approval queue, for Approvers and Org Admins. No user may approve their own request, a rule enforced in the approval policy rather than in the interface (Section 8.5).

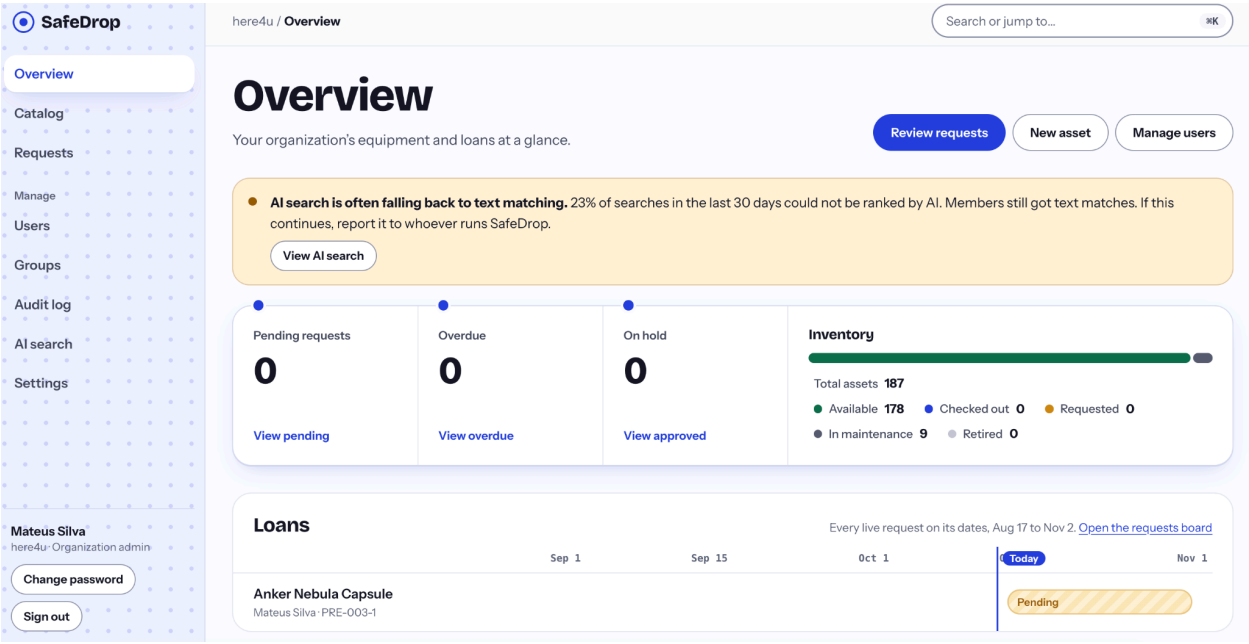


Figure 11. Admin dashboard: inventory counts and checkouts per day.

4.4 Screens not shown above - organization setup, forgotten and reset password, forced password change, request detail, members, the asset create/edit form and the not-found page - are implemented and covered by the frontend test suite, but are not captured here. Accessibility has not been formally reviewed: the interface is built from semantic elements and labelled form controls and eslint-plugin-jsx-a11y runs in CI, but no screen-reader or contrast audit has been done, and NFR-9 should not be treated as demonstrated until one is. Iteration 2 also changed screens that are not yet captured: the Organization settings screen (/admin/settings, default approval mode and pickup grace period), an Approval control on the asset form (Inherit, Required, Auto), and on request detail the borrower's "I've picked it up" and "Return this item" (condition and note) actions and the confirmer's "Confirm return" (received condition) and "Reject return" (reason) actions.

5. Database Design (if applicable)

SafeDrop stores seven collections in one MongoDB database. Every tenant-owned document carries orgId, and every index used for a tenant query begins with orgId (Section 2.3.3, NFR-4). The schema below is the one implemented in Iteration 1; field types, enumerations and the index names are those created by the migrations in code/backend/migrations. The organizations, assets and checkoutrequests rows include the Iteration 2 fields added by SCRUM-148 and SCRUM-205 (migrations 20261002000000-approval-settings and 20261004000000-pickup-settings set defaults on existing documents).

Collection	Key fields	Indexes	Notes
organizations	name, slug, approvalSettings.def	slug_unique	The tenant itself. The slug is the public

	aultMode, pickupSettings.graceHours		identifier used at login. approvalSettings.defaultMode is REQUIRED or AUTO (default REQUIRED, SCRUM-148). pickupSettings.graceHours is a whole number of hours, 0 to 720 (default 48, SCRUM-205): how long after neededFrom an approved request may wait to be collected. Migrations 20261002000000-approval-settings and 20261004000000-pickup-settings set these defaults on existing organizations, so existing behaviour did not change.
users	orgId, email, passwordHash, name, role, mustChangePassword, resetTokenHash, resetTokenExpiresAt	orgId_email_unique, orgId_role	Email is unique per organization, not globally (OD-3), which is why login also takes the org slug. passwordHash, resetTokenHash and resetTokenExpiresAt are select:false, so they are absent from query results unless a repository asks for them by name. role is one of MEMBER, APPROVER, ORG_ADMIN.
assets	orgId, name, category, description, imageUrl, retiredAt,	orgId_name, orgId_category, orgId_retiredAt	The catalogue entry (a kind of thing). Retirement is a

	approvalMode, allowedGroupIds		timestamp rather than a delete, so history stays readable. approvalMode is INHERIT, REQUIRED or AUTO (default INHERIT, SCRUM-148); INHERIT means the organization default applies.
assetunits	orgId, assetId, tag, serial, condition, status	orgId_tag_unique, orgId_assetId, orgId_status	The individual physical copy that is actually lent. condition is NEW, GOOD, FAIR or POOR. This is the level at which availability is tracked.
checkoutrequests	orgId, unitId, requesterId, state, neededFrom, neededTo, note, decidedBy, decidedAt, decisionNote, autoApproved, checkedOutAt, dueAt, reportedCondition, reportedNote, returnInitiatedAt, returnedAt, expiredAt	orgId_state_dueAt, orgId_requesterId_createdAt, orgId_unitId_state	The request-through-return lifecycle; state moves through the transitions in Section 7. autoApproved (immutable) is true when the approval policy granted the request at submission. reportedCondition, reportedNote and returnInitiatedAt hold the borrower's side of a two-step return; the unit's condition is only overwritten with what the confirmer actually received (SCRUM-205). expiredAt is set by the expiry sweep.
auditevents	orgId, actorId, actorRole, action, targetType, targetId,	orgId_timestamp, orgId_target_timestamp,	Append-only (Section 6.6). Every field is immutable, the schema is

	before, after, timestamp, requestId	orgId_actorId_timestamp	strict:"throw", and the orgId_actorId_timestamp index is what makes the audit log's member filter efficient.
refreshtokens	orgId, userId, tokenHash, familyId, expiresAt, absoluteExpiresAt, revoked	tokenHash_unique, orgId_userId, orgId_familyId, expiresAt, absoluteExpiresAt_ttl	Only the hash of a refresh token is stored. familyId links a rotated chain so that reuse of an already-rotated token can revoke the whole session; absoluteExpiresAt_ttl is a TTL index that removes expired rows.

6. Security Design

6.1 Security Principles and Threat Model

SafeDrop's security design follows three principles. First, every security decision is enforced on the server, because the browser is untrusted. Second, access is denied by default and granted only through explicit permissions. Third, tenant isolation and audit integrity are built into the structure of the system rather than depending on each developer remembering to apply them. The table below lists the threats the design addresses and where each control is described.

Threat	Example in SafeDrop	Primary controls	Req. IDs
Cross-tenant data access	A member of Organization A changes an asset ID in a request URL to read an asset owned by Organization B	Organization ID taken only from the token; tenant-scoped repositories; 404 for other tenants' IDs (6.4)	SR-2
Privilege escalation	A member calls an admin-only endpoint directly, bypassing the UI	Server-side RBAC on every route; deny by default (6.4)	SR-1
Credential attacks	Brute-force or credential-stuffing attempts against the login endpoint	bcrypt hashing; rate limiting; generic login errors (6.2)	SR-3, SR-12
Session theft and forgery	A stolen token is replayed, or a malicious site triggers a request using the victim's cookie	httpOnly Secure cookies; short-lived tokens; refresh rotation; CSRF defenses (6.2, 6.3)	SR-4, SR-5

Threat	Example in SafeDrop	Primary controls	Req. IDs
NoSQL operator injection	A login body such as `{"password": {"$ne": null}}` that matches any password	Schema validation with strict types; filter sanitization (6.5)	SR-6
Stored XSS	A script saved in an asset name executes when an admin views the inventory	React output escaping; no raw HTML rendering; Content Security Policy (6.5)	SR-6
Audit tampering and repudiation	An admin deletes a return record, or a member disputes that a return was recorded	Append-only repository; insert-only database role; server timestamps; atomic writes (6.6)	SR-8, SR-9, SR-10
Secret leakage	The JWT signing key or database connection string is committed to GitHub	Environment variables; secret scanning (6.8)	SR-11
Vulnerable dependencies	An npm package with a known vulnerability ships to production	npm audit and CodeQL in CI; OWASP ZAP on staging (6.8)	SR-13

6.2 Authentication

Decision. SafeDrop implements its own email-and-password authentication using bcrypt and JSON Web Tokens (JWTs) [5], rather than a hosted identity provider such as Clerk or Auth0. This gives the team full control over the tenant and role claims carried in each session and over cookie configuration, avoids a third-party dependency and its cost, and keeps authentication within the course's learning objectives. The trade-off is that the team owns the correctness of security-critical code. That risk is reduced by using well-maintained libraries for all cryptographic operations (for example, `bcrypt` and `jsonwebtoken`), never writing cryptographic primitives by hand, and requiring automated tests for every authentication path.

Password storage

- Passwords are hashed with bcrypt, which generates a unique salt for every hash and stores it with the hash (SR-3) [4]. The cost factor starts at 12 and is tuned so that hashing takes a noticeable fraction of a second on the production host.
- Password length rules follow current NIST SP 800-63B guidance [11]. The maximum length is capped at 72 bytes, because bcrypt ignores input beyond that length.
- Request logging redacts password fields, so plaintext passwords never appear in logs (SR-3).

Login flow

1. The SPA submits the email and password over HTTPS to the login endpoint, which is rate-limited (SR-12).
2. The request body is validated, and the user record is looked up. Login takes the organization’s slug together with the email address, because email is unique per organization rather than across SafeDrop (OD-3).

3. The submitted password is compared with the stored hash. Every failure returns the same generic message and status code. When no matching user exists, the server still performs a comparison against a dummy hash, so response timing does not reveal which email addresses have accounts.
4. On success, the server issues an access token and a refresh token and sets both as `HttpOnly` and `Secure` cookies. The refresh-token cookie is restricted to the refresh endpoint's path, so it is not sent with ordinary API requests.
5. Successful and failed login attempts are recorded in the application's security log, without passwords or token values.

Token design

Token	Contents	Lifetime	Storage	Revocation
Access token (JWT)	User ID (`sub`), organization ID, role, issued-at, expiry	Short; proposed 15 minutes	HttpOnly, Secure cookie	Expires on its own; not individually revocable
Refresh token	Opaque random value, not a JWT	Idle timeout of 30 minutes (SR-4); proposed absolute maximum of 12 hours	HttpOnly, Secure cookie; stored only as a hash in the database	Deleted on logout; rotated on every use; reuse of an already-rotated token revokes the entire session

- Access tokens are signed with a secret key loaded from an environment variable (SR-11). Verification pins the expected signing algorithm and rejects tokens that declare any other algorithm, including "none".
- The 30-minute inactivity timeout in SR-4 is implemented through the refresh token. If the SPA does not refresh within 30 minutes, the session ends and the user is returned to the login screen.
- On logout, the server marks the stored refresh-token record revoked and clears both cookies; the record itself is removed later by the collection's expiry index. The effect is the same - the token cannot be used again - but the row survives long enough to detect reuse of an already-revoked token.

Trade-off: stale roles. Because the role is embedded in the access token, a user whose role is reduced keeps their previous permissions until that token expires, which is at most 15 minutes. This is acceptable for most actions. For user-management actions, such as changing a user's role, the service re-reads the actor's current role from the database rather than trusting the token. Note the limit of this as implemented in Iteration 1: approving or denying a request, and recording checkout and return, still trust the role in the token, so a demoted approver keeps that authority until the access token expires - at most 15 minutes.

6.3 Session Cookies Across Domains (OD-1)

SR-4 requires httpOnly cookies, which protect tokens from being read by injected scripts. However, the SPA on `*.vercel.app` and the API on `*.onrender.com` are on different registrable domains, so browsers treat the API's cookie as a third-party cookie. This creates two problems. First, the cookie must be marked `SameSite=None`, which gives up SameSite's built-in protection against cross-site request forgery (CSRF). Second, browsers that restrict third-party cookies may drop the cookie entirely, causing login to fail silently for some users and threatening the browser-compatibility requirement (NFR-8). The team must choose one of the following options.

Option	How it works	Advantages	Disadvantages
A. Proxy through Vercel	The SPA calls `/api/*` on its own origin, and a Vercel rewrite forwards those requests to Render	Cookies are first-party, so `SameSite=Lax` or `Strict` works; no cost	Adds a network hop; depends on hosting configuration; the Render URL remains directly reachable, so CORS still matters
B. Custom domain	Frontend and API share one registrable domain, for example `app.` and `api.` subdomains	Same-site cookies; clean, professional URLs	Requires purchasing a domain and configuring DNS on both hosts
C. Cross-site cookies	Keep separate domains and set `SameSite=None; Secure`	No infrastructure changes	May be blocked by browser privacy protections; requires a separate CSRF token mechanism

Decision. Option A for Iteration 1: the SPA calls /api/* on its own origin and a Vercel rewrite forwards those requests to Render, so cookies are first-party and SameSite=Lax applies. The team will move to Option B if it acquires a domain. Under any option, state-changing endpoints accept only application/json request bodies and verify the Origin header against the allowlist, which provides defense in depth against CSRF [4].

6.4 Authorization and Tenant Isolation

Role-based access control

SafeDrop defines three roles in Iteration 1: Member, Approver, and Org Admin (SR-1). The matrix below reflects Iteration 2 (SCRUM-148 and SCRUM-205). Each change of custody is recorded by the side that can verify it.

Action (within the user's own organization)	Member	Approver	Org Admin
Browse available assets	Yes	Yes	Yes
Submit a checkout request	Yes	Yes	Yes
View own requests and borrowing history	Yes	Yes	Yes
Approve or deny a request (never one's own)	No	Yes	Yes

Action (within the user's own organization)	Member	Approver	Org Admin
Record pickup of an approved request	Own Request Only	Yes	Yes
Start a return and report condition	Own Request Only	Yes	Yes
Confirm or reject a return (never one's own, except sole confirmer)	No	Yes	Yes
Configure approval mode and pickup grace period	No	No	Yes
Add, edit, or retire assets	No	No	Yes
Invite users and assign roles	No	No	Yes
View the organization's audit log	No	No	Yes

- **Separation of duties.** No user may approve their own request, regardless of role. This rule is enforced in the approval policy (Section 8.5), not in the user interface.
- **Separation of duties on returns (SCRUM-205).** No user may confirm or reject the return of their own loan, so an approver cannot mark an item back while it is still in their bag. The one exception is an organization whose only Approver or Org Admin is the requester: nobody else could ever close the loan, so it is allowed and the audit entry carries selfConfirmed: true. Like the approval rule, this lives in the approval policy (canConfirmReturn, Section 8.5). Recording pickup is deliberately not split by permission: the route is open to every role and the service admits the requester or a holder of requests:handoff, because giving members requests:handoff would also let them confirm returns.
- **Deny by default.** Each route must declare the permission it requires, and the authorization middleware rejects any route that has none. An automated test verifies that every registered route declares a permission, so a newly added route cannot be left unprotected by accident.

Tenant isolation

- **Single source of tenant identity.** The organization ID comes only from the verified access token. It is never read from the URL, query string, or request body.
- **Repository contract.** Every repository function takes the organization ID as its first parameter and includes it in every query filter, including updates and deletes. A missing filter is therefore visible in code review as a function called without its required argument.
- **Server-assigned ownership.** When a document is created, the organization ID is set by the server, never copied from the request body. This prevents mass-assignment attacks.
- **No existence leaks.** A request for a resource that belongs to another organization returns 404 Not Found rather than 403 Forbidden, so attackers cannot confirm that the ID exists.

- **Verification.** For every resource endpoint, an automated test seeds two organizations and confirms that a token from one organization receives 404 for the other organization's IDs.

```
// src/repositories/assetRepository.js (illustrative)
async function findAssetById(orgId, assetId) {
  return Asset.findOne({ _id: assetId, orgId });
}
```

6.5 Input Validation and Output Encoding

- **Schema validation (SR-6).** Every route declares a schema for its path parameters, query parameters, and body. The validation middleware rejects requests that do not match with 400 Bad Request, and it rejects unknown fields. Validation uses Zod (OD-6), one library for the whole team.
- **NoSQL operator injection.** Strict typing is the primary defense: a field declared as a string cannot arrive as an object such as `{"$ne": null}`. As defense in depth, Mongoose's `sanitizeFilter` option is enabled, raw request objects are never passed directly into queries, and server-side JavaScript operators such as `$where` are never used.
- **Stored XSS.** React escapes rendered text by default. Rendering user-supplied content with `dangerouslySetInnerHTML` is prohibited and included in the code review checklist. Text fields have maximum lengths, and the frontend's hosting configuration sets a Content Security Policy header that restricts where scripts may load from.
- **Safe error responses.** Production error responses contain a status code and a generic message only, never stack traces, query details, or database error text.

6.6 Audit Log Integrity

The audit log is the product's evidence of record for disputes, so its integrity is protected at both the application layer and the database layer. Each audit entry has the following structure.

Field	Purpose
`orgId`	Tenant scope; audit entries are visible only within their organization
`actorId`, `actorRole`	Who performed the action, and the role they held at that moment (SR-10). One action has no human actor: REQUEST_EXPIRED is written by the expiry sweep with actorId null and actorRole SYSTEM, because nobody decided it

Field	Purpose
`action`	The event type. Iteration 1 defines thirteen, enumerated in code and enforced by the schema: REQUEST_SUBMITTED, REQUEST_APPROVED, REQUEST_DENIED, REQUEST_CANCELLED, ASSET_CHECKED_OUT, ASSET_RETURNED, ASSET_CREATED, ASSET_UPDATED, ASSET_RETIRED, USER_ROLE_CHANGED, USER_INVITED, USER_PASSWORD_RESET, ORG_CREATED. Adding one is a design change and updates this list. Iteration 2 adds REQUEST_AUTO_APPROVED and ORG_SETTINGS_UPDATED (SCRUM-148), and REQUEST_EXPIRED, RETURN_INITIATED and RETURN_REJECTED (SCRUM-205). REQUEST_AUTO_APPROVED is separate from REQUEST_APPROVED so the log never shows an approval with no approver behind it; its after-state names the policy and has decidedBy: null. ASSET_CHECKED_OUT carries selfReported: true when the borrower recorded their own pickup, and ASSET_RETURNED stores the reported and received condition and selfConfirmed.
`targetType`, `targetId`	The record the action affected. One of five target types: Organization, User, Asset, AssetUnit, CheckoutRequest
`before`, `after`	Snapshots of the changed fields (SR-9). Password hashes, tokens, and other secrets are always excluded.
`timestamp`	Generated by the server; never accepted from the client
`requestId`	Correlates the audit entry with application logs when investigating an incident

- **Atomic writes (SR-9, NFR-2).** The service performs the state change and appends the audit entry within one MongoDB transaction [6]. If either write fails, both roll back, so a state change can never exist without its audit entry. This replaces the Iteration 0 middleware approach (OD-2, decided).
- **Application-layer immutability (SR-8).** The audit repository exposes only append and query functions. There are no update or delete functions and no update or delete API routes for audit entries.
- **Database-layer immutability (SR-8) - defined, not yet applied. The API's database user is to be assigned a custom Atlas role granting only find and insert privileges on the audit collection [7]. As of Iteration 1 the role is specified and a test is staged (code/docs/audit-db-role.md; the test runs only against a real authenticated deployment), but the role has not been created in the Atlas console and the API user has not been reassigned to it (SCRUM-125). Until that is done this layer provides no protection and SR-8 rests on the application layer alone - which matters, because the application-layer guard is enforced by Mongoose, so a raw driver call that bypasses Mongoose bypasses it too. Doing this requires granting read and write privileges collection by collection, since a database-wide read-write role would also cover the audit collection.**
- **Restricted reading.** Only Org Admins can read the audit log, and results are scoped to their organization and paginated.

- **Planned tamper evidence (Iteration 3).** Each entry will store a hash of its contents combined with the previous entry's hash. Any modification made with direct database access would then break the chain and be detectable.

6.7 Transport Security and HTTP Headers

- **HTTPS everywhere (SR-5).** Vercel and Render terminate TLS for all traffic. The API sets HTTP Strict Transport Security (HSTS), all cookies are marked `Secure` so they are never sent over plain HTTP, and connections to Atlas use TLS.
- **Security headers.** Standard headers, including HSTS, `X-Content-Type-Options`, and frame-embedding protections, are applied by the Helmet middleware (OD-6) [8].
- **CORS (SR-14).** Allowed origins come from an environment variable configured per environment. Credentialed requests are permitted only from allowlisted origins, and the API never echoes back an arbitrary `Origin` value. If Option A is chosen in Section 6.3, browser traffic becomes same-origin, but CORS remains in place to protect against direct calls to the Render URL.

6.8 Secrets Management and Supply Chain

- **Secrets (SR-11).** The JWT signing key, Atlas connection string, and later the Microsoft Foundry keys are stored in the Render and Vercel environment settings and in GitHub Actions secrets. The `.env` file is listed in `.gitignore`, and a committed `.env.example` documents variable names without values. Each environment has its own secrets.
- **Secret scanning.** GitHub secret scanning and push protection are enabled if they are available for the repository.
- **Dependency and code scanning (SR-13).** npm audit and CodeQL run on every pull request. The proposed policy is that high or critical npm audit findings fail the build.
- **Dynamic scanning.** An OWASP ZAP scan runs against staging before each release is promoted to production.

6.9 Data Protection and Privacy

- **Minimal personal data (SR-7).** Iteration 1 stores names, email addresses, organization membership, and borrowing history. It stores no additional personal data, so SR-7's field-level encryption requirement is not triggered. Atlas encrypts stored data at rest by default.
- **Borrowing history (NFR-14).** For educational customers, borrowing history may be student data. Access is limited to the borrower and their organization's Org Admins.
- **Logs.** Application logs exclude passwords, token values, and cookie contents.
- **AI features (Iteration 3).** Only the minimum fields needed for each AI feature are sent to Microsoft Foundry, and names and email addresses are never included.

6.10 Security Requirements Traceability

ID	Design element	Section	How it will be verified
SR-1	RBAC middleware; route permission map; deny by default	2.5, 6.4	Role-based route tests; test that every route declares a permission
SR-2	Tenant ID from token only; tenant-scoped repositories	6.4	Cross-tenant tests for each resource endpoint. Implemented: all 11 rows of tests/integration/security/tenantIsolation.test.js are active (6 endpoints that were still flagged unimplemented were enabled in Iteration 1); each asserts 404, not 403.
SR-3	bcrypt with per-hash salt; password redaction in logs	6.2	Unit tests; code review
SR-4	Short-lived JWT, rotating refresh token, httpOnly Secure cookies, 30-minute idle timeout	6.2, 6.3	Integration tests for expiry, rotation, and token reuse
SR-5	HTTPS, HSTS, Secure cookies, TLS to Atlas	6.7	Header checks; OWASP ZAP scan
SR-6	Schema validation; filter sanitization; React escaping; CSP	6.5	Injection and XSS test cases; OWASP ZAP scan
SR-7	No additional PII stored; Atlas encryption at rest	6.9	Data model review each iteration
SR-8	Append-only audit repository (implemented); insert-only database role (defined, not yet applied)	6.6	PARTIALLY VERIFIED. Application layer: the repository exposes only append and query and the model throws on every mutating Mongoose operation - covered by tests. Database layer: the role is defined in code/docs/audit-db-role.md and its test is staged but skipped, because it needs a real authenticated deployment. SR-8 is not fully satisfied until the role is applied (SCRUM-125).
SR-9	Transactional audit write in the service layer	2.5, 6.6	Tests asserting one audit entry per state change; rollback test
SR-10	Actor, role, server timestamp, before/after state, request ID	6.6	Review of sample audit entries for a full request lifecycle
SR-11	Environment variables; per-environment secrets; secret scanning	6.8	Secret scanning results; code review
SR-12	Rate limiting on the authentication routes, and on the public organization bootstrap route	2.5, 6.2	Test that exceeding the limit returns 429. Both limiters covered: the auth limiter skips successful requests; the organization-create limiter deliberately does not, because a successful anonymous creation is the abuse (SCRUM-114).
SR-13	npm audit and CodeQL in CI	6.8	CI results on every pull request
SR-14	CORS allowlist per environment	6.7	Test that a disallowed origin is rejected

7. Business Logic and/or Key Algorithms

The checkout request state machine is the central algorithm in Iteration 1. A single transition table is the authority on what a request may do next, and every handler goes through it rather than testing the state inline, so an illegal move is refused in one place instead of in eight. Each transition also carries what it does to the physical unit, which is what keeps a request's state and its unit's availability from drifting apart. Every transition below is applied inside one MongoDB transaction together with its audit entry (Sections 2.5 and 6.6), so a request can never change state without the matching audit record. Submitting a request is the one move with no row in the table: it creates the request rather than transitioning an existing one, moving a unit from AVAILABLE to REQUESTED. Iteration 2 added two states: EXPIRED, for an approval nobody collected, and RETURN_PENDING, for a return the borrower has started but nobody else has confirmed (SCRUM-205). The table lives in services/checkout.service.js (TRANSITIONS), with a frontend mirror in utils/requestState.js; the two change together. Each write is also a compare-and-set on the expected current state, so when two people act at once the second gets 409 instead of a double transition.

From	To	Unit becomes	Triggered by
PENDING	APPROVED	HELD	POST /requests/:id/approve (requests:decide, never the requester)
PENDING	DENIED	AVAILABLE	POST /requests/:id/deny
PENDING	CANCELLED	AVAILABLE	POST /requests/:id/cancel (the requester withdraws)
APPROVED	CHECKED_OUT	OUT	POST /requests/:id/checkout, by the borrower or a requests:handoff holder; stamps dueAt from neededTo
APPROVED	CANCELLED	AVAILABLE	POST /requests/:id/cancel
APPROVED	EXPIRED	AVAILABLE	Expiry sweep: neededFrom plus the grace period has passed (POST /requests/expire-approvals)

CHECKED_OUT	RETURN_PENDING	(unchanged)	POST /requests/:id/initiate-return (the requester, with the condition they report); the unit stays OUT
CHECKED_OUT	RETURNED	AVAILABLE	POST /requests/:id/return (walk-in return, by an Approver or Org Admin who is not the requester)
CHECKED_OUT	OVERDUE	(unchanged)	The due date passes (POST /requests/mark-overdue); the unit stays OUT because it is still out
CHECKED_OUT	LOST	RETIRED	The unit is written off (transition exists; no endpoint yet)
OVERDUE	RETURN_PENDING	(unchanged)	POST /requests/:id/initiate-return
OVERDUE	RETURNED	AVAILABLE	POST /requests/:id/return - a late return is still a return
OVERDUE	LOST	RETIRED	The unit is written off (transition exists; no endpoint yet)
RETURN_PENDING	RETURNED	AVAILABLE	POST /requests/:id/return (confirm, recording the condition actually received)
RETURN_PENDING	CHECKED_OUT	(unchanged)	POST /requests/:id/reject-return (with a reason)

DENIED, CANCELLED, RETURNED, EXPIRED and LOST are terminal: they have no outgoing transitions.

7.1 Submission and Auto-Approval (SCRUM-148)

Submitting a request is the one move with no row in the table, because it creates the request rather than transitioning one. Inside the same transaction, submit() loads the unit's asset and the organization and asks the policy `policyFor(org).requiresApproval({ request, asset, org })`. If approval is required, the unit moves from AVAILABLE to REQUESTED and the request is created PENDING, exactly as in Iteration 1. If it is not, the unit moves from AVAILABLE to HELD, the request is created APPROVED with autoApproved: true, and two audit entries are written: REQUEST_SUBMITTED and REQUEST_AUTO_APPROVED. The mode is evaluated only at submission, so changing a setting never re-decides requests that are already PENDING (SCRUM-148 AT-4).

7.2 Two-Step Return and Expiry (SCRUM-205)

RETURN_PENDING is the borrower's word that the item is back, waiting for someone else's. The unit stays OUT through it, and the borrower's reported condition is kept on the request rather than written to the unit, because a claim is not evidence that the item arrived. A different Approver or Org Admin then either confirms, recording the condition they actually received (that value is what the unit gets), or rejects with a reason, which returns the request to CHECKED_OUT. A rejected return goes back to CHECKED_OUT even if it had been OVERDUE; the next overdue sweep flags it again from its unchanged dueAt. Walk-in returns, where staff record the return directly from CHECKED_OUT or OVERDUE, still work.

An approval nobody collects would otherwise hold its unit indefinitely. The expiry sweep moves every APPROVED request whose neededFrom plus the organization's pickupSettings.graceHours (default 48) has passed to EXPIRED and frees its unit. There is no scheduler yet, so, like mark-overdue, an Org Admin runs it on demand (POST /requests/expire-approvals) against the server's clock. Unlike mark-overdue it is audited, because it changes custody: each request is its own transaction, in which the state change, the unit release and the REQUEST_EXPIRED entry commit together, and a request picked up or cancelled mid-sweep fails its compare-and-set and is skipped.

8. Design Patterns

This section describes the design patterns SafeDrop uses in Iteration 1, plus one pattern whose Iteration 1 implementation was chosen specifically to support Iteration 2. Only patterns that are reflected in the codebase are listed, so reviewers can verify each one against the implementation. Iteration 2 has now used that pattern as intended (Section 8.5).

Pattern	Category	Where it is used	Iteration
Layered Architecture	Architectural	Backend folder structure	1
Service Layer	Enterprise application [9]	`src/services`	1
Repository	Enterprise application [9]	`src/repositories`; frontend `src/services`	1
Chain of Responsibility	Behavioral [10]	Express middleware pipeline	1
Strategy	Behavioral [10]	Approval policy (services/policies/approvalPolicy.js)	1 (fixed policy); 2 (configurableApproval and canConfirmReturn, implemented)
Provider and Custom Hooks	React	`src/context`, `src/hooks`	1

8.1 Layered Architecture

Problem. In a small Express application, route handlers often grow to contain HTTP parsing, business rules, and database queries in the same function, which makes them difficult to test and to change safely.

Solution in SafeDrop. The backend is divided into routes, controllers, services, repositories, and models (Section 2.3.2). Each layer depends only on the layers below it.

Consequences. Each layer can be tested with the layer beneath it mocked, and the fan-out metric in the QA plan stays low. The cost is some pass-through code for simple operations, which the team accepts in exchange for consistency.

8.2 Service Layer

Problem. Workflows such as approving a request touch several records, must enforce business rules, and must be atomic with their audit entry. Placing that logic in controllers would couple it to HTTP.

Solution in SafeDrop. Services define the application's operations, for example `submitRequest`, `approveRequest`, `recordCheckout`, and `recordReturn`. Each service coordinates repository calls, applies the approval policy, and owns the database transaction that includes the audit entry (Section 6.6).

Consequences. Controllers stay thin. The same service functions can later be reused from entry points other than HTTP, such as a scheduled job that sends overdue-return reminders.

8.3 Repository

Problem. Scattering database queries across the codebase makes it hard to guarantee that every query is scoped to a tenant, and hard to mock the database in unit tests.

Solution in SafeDrop. All database access goes through repository functions that hide the underlying queries. The SPPP coding standard's `getUser` example [1] reflects this pattern. SafeDrop extends it by requiring the organization ID as every function's first parameter (Section 6.4). On the frontend, `src/services` plays the same role for API calls.

Consequences. Tenant scoping is enforced in one place and is easy to check in code review, and services can be tested with in-memory mock repositories. Replacing the ODM later would affect only the repository and model layers.

8.4 Chain of Responsibility

Problem. Every request must pass through the same sequence of security checks, and any check must be able to stop the request.

Solution in SafeDrop. Express middleware is a variant of Chain of Responsibility. Each handler either passes the request along by calling `next()` or ends processing by responding or raising an error. SafeDrop's chain is described in Section 2.5.

Consequences. New cross-cutting concerns can be added without modifying controllers. However, the order of handlers is significant; for example, authorization must follow authentication. The order is therefore defined in one place, `src/app.js`, and covered by tests.

8.5 Strategy (Approval Policy)

Problem. Iteration 1 requires a simple, fixed approval step. Iteration 2 requires approval rules that administrators can configure globally, per item, or per user tier. Writing the Iteration 1 rule directly into the request service would mean rewriting that service in Iteration 2.

Solution in SafeDrop. The request service depends on an approval policy interface rather than on a specific rule. Iteration 1 provides a single fixed policy: every request requires approval by an Approver or Org Admin who is not the requester. In Iteration 2, additional policies, such as automatic approval for low-risk items or tier-based rules, implement the same interface and are selected according to organization or item configuration.

```
// Iteration 1: a single fixed policy (illustrative)
const fixedApprovalPolicy = {
  requiresApproval(request) {
    return true;
  },
  canApprove(approver, request) {
    return ['APPROVER', 'ORG_ADMIN'].includes(approver.role)
      && approver.id !== request.requesterId;
  },
};

// The service depends on the policy interface, not on a concrete rule
async function approveRequest(ctx, requestId, policy = fixedApprovalPolicy) {
```

```
  // load request (tenant-scoped), check policy.canApprove,
  // update status and append audit entry in one transaction
}
```

Iteration 2: how the policy absorbed the new modes

What was built. SCRUM-148 needed an organization-wide default that an individual asset can override, so cheap items go straight out while expensive ones still get a human decision. SCRUM-205 needed the same separation of duties at the end of a loan, so nobody can confirm their own return. Both were added inside the approval policy. In the code, the illustrative canApprove above is named canDecide and approveRequest is approve() in services/checkout.service.js. A policy now answers three questions:

Method	Question it answers	Called from
requiresApproval({ request, asset, org })	Does this request need a human decision?	submit()
canDecide(request, actor)	May this actor approve or deny it?	approve(), deny()
canConfirmReturn(request, actor, org, { otherConfirmers })	May this actor confirm or reject its return?	returnUnit(), rejectReturn()

Iteration 2 added a second policy, configurableApproval, and policyFor(org) now returns it for every organization. It changes only whether a human decides, never who: its canDecide is the Iteration 1 function, reused unchanged.

```
// services/policies/approvalPolicy.js (excerpt)
export const configurableApproval = Object.freeze({
  name: 'configurable-approval',
  requiresApproval({ asset, org } = {}) {
    return effectiveApprovalMode(asset, org) !== APPROVAL_MODE.AUTO;
  },
  canDecide: requireDistinctApprover.canDecide,  // who decides: unchanged
  canConfirmReturn,                              // SCRUM-205
});

// asset override, else org default, else REQUIRED
export function effectiveApprovalMode(asset, org) { ... }

export function policyFor(_org) {
  return configurableApproval;
}

// services/checkout.service.js, submit(): the one call site that changed
const policy = policyFor(org);
const autoApprove = !policy.requiresApproval({ request: input, asset, org });
```

Resolving the mode. effectiveApprovalMode applies three rules in order: an asset's approvalMode wins unless it is INHERIT; otherwise the organization's approvalSettings.defaultMode applies; anything missing or unrecognized resolves to REQUIRED. Failing closed matters, because an unknown or corrupt value that quietly auto-approved would hand every member a free pass. The migrations set existing documents to REQUIRED and INHERIT, so until an admin changes a setting the new

policy behaves exactly like the Iteration 1 one.

Asset approvalMode	Organization defaultMode	Result
AUTO	REQUIRED	Auto-approved (SCRUM-148 AT-1)
REQUIRED	AUTO	Needs approval (AT-2)
INHERIT	AUTO or REQUIRED	Follows the organization
Missing or unknown value	Any	Needs approval (fail closed)

How the service absorbed it. approve() did not change: it still loads the request, asks policyFor(org).canDecide(), and commits the transition with its audit entry. The only call site that changed is submit(), which now passes the asset and organization to requiresApproval (the Iteration 1 interface passed only the request, which is not enough once the rule is configurable) and, when the answer is no, creates the request directly in APPROVED (Section 7.1). Org Admins change the rules through PATCH /api/organizations/me/approval-settings (org:settings) and the asset's approvalMode field (assets:write); both are Org Admin only, and each change is audited as ORG_SETTINGS_UPDATED or ASSET_UPDATED with before and after values.
Returns use the same seam (SCRUM-205). canConfirmReturn is the return-side twin of canDecide: the actor needs requests:handoff and must not be the requester. The one exception is the sole-confirmer fallback: when no other active Approver or Org Admin exists, the requester is allowed and the answer carries selfConfirmed: true, which the service writes into the audit entry. A policy must not read the database, so the service counts the other confirmers and passes the number in; an unknown count is treated as "someone else exists", so the check fails closed.

Consequences. Iteration 2 adds new policies without modifying `approveRequest`, and each policy can be unit-tested in isolation. The separation-of-duties rule (Section 6.4) lives in exactly one place. In practice: SCRUM-148 and SCRUM-205 arrived as policy code plus one changed call site rather than as edits to the approval workflow, and the separation-of-duties rules for both approving and confirming live in one file. registerPolicy() validates a new policy's shape at registration, so a per-group or per-tier policy can be added the same way. The cost is that requiresApproval's argument grew from the request alone to { request, asset, org }, and a future policy that needs more context (for example the requester's group) would grow it again. Known gap: an organization with a single admin still cannot approve that admin's own request unless auto-approval applies; the team agreed to handle that sole-approver fallback in a separate story.

8.6 Provider and Custom Hooks (Frontend)

Problem. Many screens need the signed-in user's identity, role, and organization, and many screens load data from the API. Passing that state manually through component props, or repeating data-loading logic in each screen, leads to duplication and tightly coupled components.

Solution in SafeDrop. A context provider in `src/context` supplies the current user, role, and organization to the entire component tree. Custom hooks in `src/hooks` encapsulate data loading and submission by calling `src/services`, so page components focus on presentation.

Consequences. Components can be tested by wrapping them in a test provider and mocking hooks or services. Context should be reserved for truly application-wide state, because any change to a context value re-renders all components that consume it.

9. Rest APIs

The API is the only component with database access, and every route below is declared in code/backend/src/routes with the permission it requires. Routes are declared with a permission, never a role; a route that declares none is refused when the route table is built, which is how deny-by-default (SR-1) is enforced structurally rather than by convention. An automated test walks the built route table and checks every route against the permission matrix. All paths are relative to /api, and every response is JSON. Five routes are public; the rest require a valid access token, are scoped to the caller's organization, and return 404 rather than 403 for another tenant's id (SR-2).

Method and path	Permission	Purpose
POST /auth/login	public	Exchange org slug, email and password for session cookies. Rate-limited (SR-12).
POST /auth/refresh	public	Rotate the refresh token and issue a new access token.
POST /auth/forgot-password	public	Request a reset link. Always answers the same way, whether or not the address exists.
POST /auth/reset-password	public	Complete a reset using the mailed single-use token.
POST /organizations	public	Bootstrap a new organization and its first Org Admin. Capped at 5 per IP per hour, counting successes (SCRUM-114).
GET /auth/me	session:self	The signed-in user, role and organization.
POST /auth/logout	session:self	Revoke the session and clear both cookies.
POST /auth/change-password	password:self	Change one's own password; ends every other session.

GET /assets	assets:read	Browse the catalogue. Paginated; filters by category and retirement.
GET /assets/:id	assets:read	One asset with its units.
POST /assets	assets:write	Create an asset. Accepts approvalMode (INHERIT, REQUIRED or AUTO).
PATCH /assets/:id	assets:write	Edit an asset. Accepts approvalMode (INHERIT, REQUIRED or AUTO).
POST /assets/:id/retire	assets:write	Retire an asset. Refused while any unit is still out.
POST /assets/:id/units	assets:write	Add a physical unit to an asset.
GET /assets/:id/history	audit:read	The chain of custody for one asset (SCRUM-29). Behind audit:read rather than assets:read, because what it returns is the audit trail reached by a different question. Unions the events for three target types - the Asset, its AssetUnits and the CheckoutRequests against those units - in one query, paginated once.
GET /requests	requests:read:own	List requests visible to the caller. scope=others gives the organization-wide view without the caller's own requests (the approval queue).
GET /requests/:id	requests:read:own	One request.
POST /requests	requests:create	Submit a checkout request. Created PENDING, or APPROVED when the approval policy auto-approves (Section 7.1).
POST /requests/:id/cancel	requests:create	Withdraw one's own pending request.
POST /requests/:id/approve	requests:decide	Approve a request. Never one's own (separation of duties, Section 8.5).

POST /requests/:id/deny	requests:decide	Deny a request, with a reason.
POST /requests/:id/checkout	requests:create	Record pickup. Open to every role at the route; the service admits only the requester or a requests:handoff holder (403 for another member). 409 if already checked out.
POST /requests/:id/initiate-return	requests:create	The requester starts a return and reports the condition. 404 for anyone else.
POST /requests/:id/return	requests:handoff	Confirm a pending return, or record a walk-in return, with the received condition. Never one's own, except the sole confirmer.
POST /requests/:id/reject-return	requests:handoff	Reject a pending return with a reason; back to CHECKED_OUT. Same confirmer rule.
POST /requests/expire-approvals	requests:expire	Org Admin. Run the expiry sweep against the server's clock; no body.
GET /users	users:manage	List the organization's members.
POST /users/invite	users:manage	Add a member with an initial password they must change.
PATCH /users/:id/role	users:manage	Change a member's role. Re-reads the actor's role from the database.
POST /users/:id/password	users:manage	Set a member's password and revoke their sessions.
GET /audit	audit:read	The organization's audit log. Paginated; filters by member (actor), action, target type and date range.
GET /dashboard/summary	dashboard:read	Counts and recent activity for the admin dashboard.
GET /organizations/me/approval-settings	org:settings	Read the organization's default approval mode.

PATCH /organizations/me/approval-settings	org:settings	Set defaultMode (REQUIRED or AUTO). Audited as ORG_SETTINGS_UPDATED; not retroactive.
GET /organizations/me/pickup-settings	org:settings	Read the pickup grace period.
PATCH /organizations/me/pickup-settings	org:settings	Set graceHours (whole hours, 0 to 720). Audited as ORG_SETTINGS_UPDATED.

10. Outside /api, GET /health is a public probe: 200 when the API and its database are up, 503 when the database is unreachable.

11. Any Additional Topics you would like to include.

10.1 Open Design Decisions

The following design decisions remain open. Each is referenced by its OD number where it affects this document. When a decision is made, record the outcome in the Status column and in the revision history, then update the affected sections.

ID	Decision needed	Options	Recommendation	Affects	Status
OD-1	How the SPA and API share authentication cookies across domains	A: Vercel proxy B: Custom domain C: Cross-site cookies with CSRF tokens	A for Iteration 1; B if a domain is acquired	2.4, 6.3	DECIDED: Option A (Vercel proxy). Implemented via a Vercel rewrite; cookies are first-party and SameSite=Lax. Option B remains the upgrade path if a domain is acquired.
OD-2	Where the audit entry is written (changes the Iteration 0 design)	Trailing middleware; or inside the service transaction	Service transaction	2.5, 6.6	DECIDED: inside the service transaction. Implemented; the audit write and the state change commit or roll back together, and tests prove the rollback.

ID	Decision needed	Options	Recommendation	Affects	Status
OD-3	Account and membership model	Email unique across SafeDrop or per organization; admin invitation or self-registration with an organization code; whether one person can belong to several organizations	Admin invitation for Iteration 1; settle email uniqueness before designing the schema	6.2, 6.4, Sections 3 and 5	DECIDED. Email is unique per organization; one user belongs to one organization; members are added by an Org Admin (no self-registratio n). In Iteration 1 the admin sets the initial password and shares it out of band; email delivery arrives with Forgotten Password (SCRUM-22).
OD-4	Final Iteration 1 permission matrix, including who records checkout and return handoffs	As proposed in 6.4, or a variation	Confirm with the Requirements lead against user stories	6.4	DECIDED (Iteration 2, SCRUM-205). Pickup may be recorded by the borrower or an Approver or Org Admin; a return is started by the borrower and confirmed or rejected by an Approver or Org Admin other than the requester, with a sole-confirmer fallback. Section 6.4 shows the updated matrix, and the route-table test pins it.

ID	Decision needed	Options	Recommendation	Affects	Status
OD-5	Docker's role beyond local development	Local development only; or Render also deploys from the Dockerfile	Local development and CI parity for Iteration 1	2.2, 2.3.4	DECIDED, pending team confirmation. Docker Compose for local development; CI builds both images for parity; Render deploys the API from code/backend/ Dockerfile; the SPA deploys to Vercel from source. This is broader than the original recommendation, so the team should ratify it.
OD-6	Library selections	ODM (Mongoose); validation (Joi or Zod); security headers (Helmet); rate limiting	Adopt Mongoose; choose one validation library for the whole team	2.2, 6.5, 6.7	DECIDED: Mongoose (ODM); Zod (validation, one library for the whole team - Joi is not used); Helmet (security headers); express-rate-limit (rate limiting).
OD-7	Atlas cluster tier	Free tier (no sharding, no built-in backups); or a paid tier	Free tier plus a scheduled database export job to meet the daily backup target in NFR-3	2.3.3, 6.6	DECIDED, pending team confirmation. Atlas free tier plus a scheduled export job for the daily backup in NFR-3. Per MongoDB documentation the free tier does support the custom insert-only audit role (SCRUM-125); it does not support Atlas backups or sharding. Confirm in the Atlas console before relying on it.

12. AI usage Log

(This table summarizes AI contribution in this document)

Section	Who	AI contribution (%)	Description	Verified by
1, 2, 6, 8, 10, 12, 13	Amber	80% (revise after team edits)	Claude drafted these sections and Figures 1 and 2 from the SPPP, nonfunctional requirements, and Iteration 0 deck	Alex Picard
1.1, 2.2, 2.3.3, 2.3.4, 2.4, 2.5, 3, 4, 5, 6.2, 6.6, 6.10, 7, 9, 10.1	Amber	70% (drafting; all claims verified against the repository)	Claude Code ran the verification pass this document was waiting on: compared every section against the merged Iteration 1 code, recorded the settled design decisions, corrected the four places where the document and the code disagreed, and drafted the empty sections from the implemented schema, state machine and route table. Figures 3-5 were generated from the code; Figures 6-11 are screenshots of the running application captured against the test suite's fixture data.	Alex Picard

Section	Who	AI contribution (%)	Description	Verified by
1.1, 2.6, 3, 4, 5, 6.4, 6.6, 7, 8, 8.5, 9, 10.1, 13	Orelmis	70% (drafting; claims checked against the repository)	Claude drafted the SCRUM-152 updates from the SCRUM-148 and SCRUM-205 stories and the merged code on main (approvalPolicy.js, checkout.service.js, routes, models, migrations), and merged them into this document.	Orelmis Toribio

13. References

[1] Team 3, "SafeDrop Software Project Proposal and Plan (SPPP)," CS673 Software Engineering, Boston University, Sept. 2026.

[2] Team 3, "SafeDrop Nonfunctional Requirements," CS673 Software Engineering, Boston University, Sept. 2026.

[3] Team 3, "SafeDrop Iteration 0 Presentation," CS673 Software Engineering, Boston University, Sept. 2026.

[4] OWASP Foundation, "OWASP Cheat Sheet Series" (Password Storage; Session Management; Cross-Site Request Forgery Prevention; Authorization). https://cheatsheetseries.owasp.org/

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. https://www.rfc-editor.org/rfc/rfc7519

[6] MongoDB, Inc., "Transactions," MongoDB Manual. https://www.mongodb.com/docs/manual/core/transactions/

[7] MongoDB, Inc., "Configure Custom Database Roles," MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/security-add-mongodb-roles/

[8] Express, "Production Best Practices: Security." https://expressjs.com/en/advanced/best-practice-security.html

[9] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, 2002.

[10] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

[11] National Institute of Standards and Technology, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B.

14. Glossary

Term	Definition
Access token	A short-lived JWT that proves a user's identity, organization, and role on each API request
Approval mode	Whether a checkout request needs a human decision: REQUIRED or AUTO, set per organization and overridable per asset (INHERIT uses the organization's)
Approver	A role permitted to approve or deny checkout requests within an organization
Audit entry	An immutable record of a single state-changing action, including actor, time, and before/after state
Auto-approval	A request the approval policy grants at submission with no human approver, recorded as REQUEST_AUTO_APPROVED
bcrypt	An adaptive password-hashing algorithm that includes a per-hash salt and a tunable cost factor
CI/CD	Continuous integration and continuous delivery; automated building, testing, and deployment
CORS	Cross-Origin Resource Sharing; browser rules that control which websites may call an API
CSP	Content Security Policy; an HTTP header that restricts the sources from which a page may load scripts and other resources
CSRF	Cross-Site Request Forgery; an attack in which a malicious site causes a browser to send an authenticated request
DAST	Dynamic Application Security Testing; testing a running application for vulnerabilities (for example, OWASP ZAP)
HSTS	HTTP Strict Transport Security; a header instructing browsers to use only HTTPS for a site
JWT	JSON Web Token; a signed, compact token format for carrying claims
Member	The base role for organization users who browse assets and submit checkout requests
Middleware	In Express, a function that processes a request before it reaches a route's controller
NFR	Nonfunctional requirement
NoSQL injection	An attack that inserts database query operators (such as `$ne`) into user input to alter a query's meaning
OD	Open design decision, tracked in Section 10.1
ODM	Object-Document Mapper; a library that maps application objects to database documents (for example, Mongoose)
Org Admin	The role that manages an organization's inventory, users, and roles, and can view its audit log
Organization (tenant)	A customer of SafeDrop whose data is fully isolated from all other organizations

Term	Definition
Pickup grace period	How many hours after a request's start date an approved item may wait to be collected before the expiry sweep releases it
RBAC	Role-Based Access Control; granting permissions according to a user's assigned role
Refresh token	A longer-lived, revocable credential used only to obtain new access tokens
Repository	A module that encapsulates all database operations for one kind of record
REST	Representational State Transfer; an architectural style for HTTP APIs built around resources
Return pending	A return the borrower has started but nobody else has confirmed yet; the borrower stays accountable for the item
SAST	Static Application Security Testing; analyzing source code for vulnerabilities (for example, CodeQL)
SDD	Software Design Document
SPA	Single-Page Application; a web application that runs in the browser and updates without full page reloads
SPPP	Software Project Proposal and Plan
SR	Security requirement
Stored XSS	Cross-site scripting in which malicious script is saved in the database and executed when other users view it
Tenant isolation	The guarantee that no organization can read or modify another organization's data
TLS	Transport Layer Security; the encryption protocol underlying HTTPS
Transaction	A group of database operations that either all succeed or all fail together